Vanilla policy gradient of a token y_t under train-inference mismatch (TIM) is

$$E_q[R\nabla_\theta logp(y_t|y_{<t})] = E_q[Rs_{y_t}]$$

$$= E_q[R]E_q[s_{y_t}] + Cov_q(R, s_{y_t})$$

$$= \underbrace{\bar{R}\bar{s}}_{drift} + \underbrace{E_q[(R-\bar{R})(s_{y_t}-\bar{s})]}_{signal}$$

TIM yields non-zero expected gradient even in an environment that does not provide any learning signal (e.g., R is a constant 1). That's not the case in an on-policy setting (i.e., $p = q$):
$E_p[Rs_{y_t}] = E_p[s_{y_t}] = \sum_{y_t \in V} p_{y_t} \nabla_\theta logp_{y_t} = \sum_{y_t \in V} \nabla_\theta p_{y_t} = \nabla_\theta \sum_{y_t \in V} p_{y_t} = \nabla_\theta 1 = 0$

- The covariance term captures the association between rewards and token-specific gradients. In other words, it sees which token led to which reward.
- The drift term depends only on the mean reward and does not distinguish which specific token receives which reward.

$\bar{s} = E_q[s_{y_t}] = E_q[\nabla_\theta logp_{y_t}] = \nabla_\theta E_q[logp_{y_t}] = -\nabla_\theta \mathcal{L}_{CE}(q, p)$ is the negative gradient of the cross-entropy loss. It pushes the trainer to distill from the sampler, scaled by the expected reward.

- The drift term vanishes under on-policy settings.
- The trainer can converge when the sampler is fixed.
- If the sampler is a biased copy of the trainer, each step pushes the trainer toward the sampler; the trainer's weights are then synced back to the sampler, and the error compounds in a feedback loop instead of converging

Score centering cancels the drift term by subtracting from each score the expected score under the sampler: $\tilde{s}_{y_t} = s_{y_t} - \bar{s}$

$$E_q[R\tilde{s}_{y_t}] = E_q[R]E_q[\tilde{s}_{y_t}] + Cov_q(R, \tilde{s}_{y_t})$$
$$= E_q[R]E_q[s_{y_t} - \bar{s}] + Cov_q(R, \tilde{s}_{y_t})$$
$$= E_q[R](\bar{s} - \bar{s}) + Cov_q(R, \tilde{s}_{y_t})$$
$$= Cov_q(R, \tilde{s}_{y_t})$$
$$= Cov_q(R, s_{y_t})$$

How to approximate $\bar{S}$

$\bar{s} = \sum_{y_t \in V} q_{y_t} s_{y_t}$ requires storing sampler's full next-token distribution during rollout. To save memory, scoring centering uses $\hat{q}_{y_t}$ to approximate q_{y_t}:

$$\hat{q}_{y_t} = \begin{cases} q_{y_t} & y_t \in H \\ \rho p_{y_t} & y_t \in V - H \end{cases} \quad where\ \rho = \frac{1 - \sum_{y_t \in H} q_v}{1 - \sum_{y_t \in H} p_v}$$

H is the top-k tokens, ρ is the ratio of the sampler's to the trainer's tail mass so that $\sum_{y_t \in V} \hat{q}_{y_t} = 1$.

$$E_{\hat{q}}[s_{y_t}] = \sum_{y_t \in H} q_{y_t} s_{y_t} + \rho \sum_{y_t \in V - H} p_{y_t} s_{y_t}$$
$$= \sum_{y_t \in H} q_{y_t} s_{y_t} + \rho \left(\underbrace{E_p[s_{y_t}]}_{=0} - \sum_{y_t \in H} p_{y_t} s_{y_t} \right)$$
$$= \sum_{y_t \in H} (q_{y_t} - \rho p_{y_t}) s_{y_t}$$

Notation

R: rollout reward (or advantage)
p: trainer
q: sampler ($p \neq q$ under TIM)
p_{y_t}: $p(y_t|y_{<t})$ probability of generating y_t under p
q_{y_t}: $q(y_t|y_{<t})$ probability of generating y_t under q
s_{y_t}: $\nabla_\theta logp_{y_t}$ log-probability gradient of y_t (score of y_t)
$\bar{s}$: $E_q[s_{y_t}] = \sum_{y_t \in V} q_{y_t} s_{y_t}$ expected score of token y_t under q

A concrete example of computing $E_q[Rs_{y_t}]$

Assume $y_t \in \{A, B\}, q(A) = 0.9, q(B) = 0.1$. Suppose we use two logits z_A and z_B to parameterize p's softmax, then $p(A) = \frac{e^{z_A}}{e^{z_A}+e^{z_B}}, p(B) = \frac{e^{z_B}}{e^{z_A}+e^{z_B}}$. $s_A = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}, s_B = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix}$.

When $R(A) = 1, \mathrm{R(B)} = 0$:

$$E_q[Rs_{y_t}] = \underbrace{\begin{bmatrix} 0.36 \\ -0.36 \end{bmatrix}}_{drift} + \underbrace{\begin{bmatrix} 0.09 \\ -0.09 \end{bmatrix}}_{signal} = \begin{bmatrix} 0.45 \\ -0.45 \end{bmatrix}$$

$z_A \leftarrow z_A + \eta \times 0.45, z_B \leftarrow z_B - \eta \times 0.45, \mathrm{p(A)} \uparrow, p(B) \downarrow.$

Let's exchange R $R(A) = 0, \mathrm{R(B)} = 1$:

$$E_q[Rs_{y_t}] = \underbrace{\begin{bmatrix} 0.04 \\ -0.04 \end{bmatrix}}_{drift} + \underbrace{\begin{bmatrix} -0.09 \\ 0.09 \end{bmatrix}}_{signal} = \begin{bmatrix} -0.05 \\ 0.05 \end{bmatrix}$$

$z_A \leftarrow z_A - \eta \times 0.05, z_B \leftarrow z_B + \eta \times 0.05, \mathrm{p(A)} \downarrow, p(B) \uparrow.$

When R was swapped, the signal direction reverses, but the drift direction remains unchanged.

How policy gradient reduces to SFT

$$\nabla_\theta \mathcal{L}_{SFT} = -E_{y_t \sim q \wedge I(y_t)=1}[\nabla_\theta logp(y_t|y_{<t})]$$
$$where\ I(y) = \begin{cases} +1 & y\ succeeds \\ 0 & otherwise \end{cases}$$

This is equivalent to vanilla policy gradient when R is +1/0 and the sampler q never gets updated